基于openMP和cuda的大规模空间转录组数据分析流程并行优化

学生姓名： 裴宇皓
学号：SA25006135

摘要

生物信息学是通过计算方式研究基因测序数据，从而实现生物学发现和机理解释的学科。近年来，随着单细胞及空间转录组测序技术的发展，测序数据的规模呈指数级增长，因此利用并行优化来加速计算成为必需。本实验选取了一个基本的数据处理流程，包含高变异基因筛选、PCA降维以及K-Means聚类，未使用现成python库而是使用C++逐步构建了整个数据处理流程。为加速流程的处理速度，本项目使用openMP多线程技术优化数据预处理部分，并使用CUDA异构计算架构并行加速了计算密集的PCA和k-Means部分。实验结果表明，并行方案实现了显著的加速比，极大地提升了测序数据分析的效率。

1. 项目背景与介绍

1.1 课题背景

在现代生物信息学研究中，空间转录组技术允许研究人员在保留组织空间位置信息的同时测量该位置的基因表达水平。这类数据通常具有高维、稀疏且样本量巨大的特点。本实验所使用的数据集包含 164,978 个观测点（Spots）和 18,000 个基因特征（gene），其中非零元素为 1.76 亿个。后续的基因筛选、PCA 降维以及 K-Means 聚类等整个处理流程即是在这个行数为观测点数，列数为基因数的稀疏矩阵上进行。

1.2 算法流程

本项目的基准流程主要包含以下6个模块：

1. **数据加载**：读取二进制稀疏矩阵。
2. **基因筛选**：基于方差筛选 Top 5000 高变基因。
3. **矩阵转换**：保留矩阵中的高可变基因列，并将稀疏矩阵转换为稠密矩阵方便后续计算。
4. **PCA 降维计算**：
    - 数据中心化。
    - 计算协方差矩阵。
    - 使用 Jacobi 迭代法求解特征值和特征向量。
5. **数据投影**： 将数据投影到前 50 个主成分中得到降维数据。
6. **K-Means 聚类**：对降维后的数据进行无监督聚类。

1.3 现有问题与挑战

面对如此规模的数据，传统的串行计算流程面临如下挑战：

1. **I/O 与预处理耗时**：稀疏矩阵到稠密矩阵的转换涉及大量内存操作，串行处理效率低下。
2. **计算密集型瓶颈**：PCA 降维过程中的协方差矩阵计算复杂度为 $O(N \cdot D^2)$，K-Means 聚类在每次迭代中都需要计算所有样本到质心的距离，在 CPU 上计算极其耗时。

1.3 优化目标

本项目希望通过并行计算技术实现整个流程的加速。具体采用 **CPU (OpenMP)** 处理逻辑复杂及内存密集型任务（如数据加载、高变基因筛选），采用 **GPU (CUDA)** 处理计算密集型任务（如矩阵乘法、特征分解、聚类迭代），最终实现端到端的高性能分析流程。

1.4 实验环境

- **CPU**: Intel Core i7-14650HK
- **GPU**: NVIDIA GeForce RTX 5060 (8GB, 80W)
- **OS**: Windows 11
- **Compiler**: NVCC (CUDA 12.8), MSVC (Visual Studio)

2. 流程串行实现与并行优化

本章按数据处理顺序，依次对数据加载、基因筛选、矩阵转换、PCA 降维、数据投影及 K-Means 聚类六个核心模块进行阐述。针对每个模块，首先展示基础串行算法的 C++ 实现，之后展示使用 OpenMP 或 CUDA 分别对每个模块实现并行以及优化，并展示最终实验结果。

2.1 数据加载模块

2.1.1 串行实现：

本项目使用的数据集采用 CSR 格式存储，分为元数据（Metadata）、行偏移（Indptr）、列索引（Indices）和非零值（Values）四个二进制文件。 在基础串行实现中，程序会按照顺序执行。

串行实现关键代码：

```
SparseMatrix loadFromBinary() {
    SparseMatrix mat;
    // 读取元数据
    std::ifstream
meta("D:/llm_learn_and_lab_demo_1/bio_lab/cpp_data/metadata.txt");
    if (!meta.is_open()) {
        std::cerr << "Error: Cannot open metadata.txt" << std::endl;
        exit(1);
    }
    meta >> mat.n_rows >> mat.n_cols;
    meta.close();
    // indptr
    mat.indptr.resize(mat.n_rows + 1);
    std::ifstream
indptr_file("D:/llm_learn_and_lab_demo_1/bio_lab/cpp_data/indptr.bin",
std::ios::binary);
    indptr_file.read(reinterpret_cast<char*>(mat.indptr.data()), (mat.n_rows +
1) * sizeof(int));
    indptr_file.close();
    int nnz = mat.indptr[mat.n_rows]; // 获取非零元素总数
    // 分配内存
    mat.indices.resize(nnz);
    mat.values.resize(nnz);
    // 读取 Indices
    std::ifstream
indices_file("D:/llm_learn_and_lab_demo_1/bio_lab/cpp_data/indices.bin",
std::ios::binary);
```

```
    if (indices_file.is_open()) {
        indices_file.read(reinterpret_cast<char*>(mat.indices.data()), nnz *
sizeof(int));
        indices_file.close();
    }
    // 读取 Values
    std::ifstream
data_file("D:/llm_learn_and_lab_demo_1/bio_lab/cpp_data/data.bin",
std::ios::binary);
    if (data_file.is_open()) {
        data_file.read(reinterpret_cast<char*>(mat.values.data()), nnz *
sizeof(float));
        data_file.close();
    }
    return mat;
}
```

2.1.2 并行实现：

串行 I/O 导致 CPU 在等待磁盘读取完成前处于空闲状态。由于`indices`和`values`的内存分配与读取是完全独立的，我们采用 OpenMP 的 **Section 并行**（`#pragma omp parallel sections`），将`indices`和`values`的读取分配给不同的线程同时执行。并行后总的数据加载时间缩短。

并行实现关键代码：

```
    // 并行加载 Indices 和 Values (OpenMP Sections)
#pragma omp parallel sections
    {
        // 任务 A: 读取 indices
#pragma omp section
        {
            std::ifstream
indices_file("D:/llm_learn_and_lab_demo_1/bio_lab/cpp_data/indices.bin",
std::ios::binary);
            if (indices_file.is_open()) {
                indices_file.read(reinterpret_cast<char*>(mat.indices.data()),
nnz * sizeof(int));
                indices_file.close();
            }
        }

        // 任务 B: 读取 values
#pragma omp section
        {
            std::ifstream
data_file("D:/llm_learn_and_lab_demo_1/bio_lab/cpp_data/data.bin",
std::ios::binary);
            if (data_file.is_open()) {
                data_file.read(reinterpret_cast<char*>(mat.values.data()), nnz *
sizeof(float));
                data_file.close();
            }
        }
    }
    return mat;
}
```

2.1.3 模块加速效果展示：

串行计时结果，去除第一次测量结果后计算剩余十次测量的均值，结果如下图：

```
Loading sparse matrix from binary...

>> [Benchmark] Starting: Load Sparse Matrix (Running 10 loops)...
   [Run 1] 1.920723s
   [Run 2] 1.487545s
   [Run 3] 1.498987s
   [Run 4] 1.448931s
   [Run 5] 1.409321s
   [Run 6] 1.437402s
   [Run 7] 1.446604s
   [Run 8] 1.452583s
   [Run 9] 1.457994s
   [Run 10] 1.449847s
   [Run 11] 1.459875s
-- [Result] Load Sparse Matrix
   Total (without first): 14.549088s | Avg: 1.454909s
------------------------------------------------
```

并行计时结果，去除第一次测量结果后计算剩余十次测量的均值，结果如下图：

```
Loading sparse matrix from binary...

>> [Benchmark] Starting: Load Sparse Matrix (Running 10 loops)...
   [Run 1] 1.332466s
   [Run 2] 1.154171s
   [Run 3] 1.188290s
   [Run 4] 1.183345s
   [Run 5] 1.116389s
   [Run 6] 1.304702s
   [Run 7] 1.158870s
   [Run 8] 1.137902s
   [Run 9] 1.072030s
   [Run 10] 1.063284s
   [Run 11] 1.082170s
-- [Result] Load Sparse Matrix
   Total (without first): 11.461155s | Avg: 1.146115s
------------------------------------------------

Loaded: 164978 spots x 18000 genes (NNZ: 176609566)
Data Loaded
```

可以看到使用 OpenMP 并行后，读取数据的速度加快。

2.2基因筛选模块：

2.2.1 串行实现：

串行程序遍历整个 CSR 稀疏矩阵中的所有非零元素，根据基因索引将具体的表达值及其平方值分别累加到对应的全局统计数组中，之后计算出每一个基因对应的方差。最后，创建一个索引列表，依据方差对所有基因进行降序排序，并直接截取前 5000 个基因索引作为最终的高变异特征集合.

串行实现关键代码：

```
std::vector<int> selectGenesByVariance(const SparseMatrix& sparse, int n_genes )
{
    const int n_spots = sparse.n_rows;    // spots
    const int n_genes_total = sparse.n_cols; //genes
```

```
    // 初始化
    std::vector<double> sum(n_genes_total, 0.0);    // 非零值之和
    std::vector<double> sum_sq(n_genes_total, 0.0); // 非零值平方和
    std::vector<int> nnz(n_genes_total, 0);         // 非零值数量
    // 遍历
    for (int spot_idx = 0; spot_idx < n_spots; spot_idx++) {
        const size_t start = sparse.indptr[spot_idx];
        const size_t end = sparse.indptr[spot_idx + 1];

        for (size_t j = start; j < end; j++) {
            const int gene_idx = sparse.indices[j];
            const float value = sparse.values[j];

            sum[gene_idx] += static_cast<double>(value);
            sum_sq[gene_idx] += static_cast<double>(value * value);
            nnz[gene_idx]++;
        }
    }

    // 计算真实方差
    std::vector<double> variances(n_genes_total, 0.0);
    for (int gene_idx = 0; gene_idx < n_genes_total; gene_idx++) {
        double total_sum = sum[gene_idx];
        double total_sq = sum_sq[gene_idx];
        double mean = total_sum / n_spots;
        double mean_sq = total_sq / n_spots;
        // 真实方差
        double variance = mean_sq - (mean * mean);

        if (variance < 0 && fabs(variance) < 1e-10) {
            variance = 0.0;
        }
        else if (variance < 0) {
            // check
            printf("WARNING: Negative variance for gene %d! (value=%.5f)\n",
                   gene_idx, variance);
            variance = 0.0;
        }
        variances[gene_idx] = variance;
    }

    // 创建基因索引并按方差降序排序
    std::vector<int> indices(n_genes_total);
    std::iota(indices.begin(), indices.end(), 0); // 0,1,2,...,n_genes_total-1

    std::sort(indices.begin(), indices.end(),
              [&variances](int a, int b) {
                  return variances[a] > variances[b];
              }
             );

    // 选择前n_genes个
    n_genes = std::min(n_genes, n_genes_total);
    indices.resize(n_genes);
    return indices;
}
```

2.2.2 并行实现：

使用OpenMP对原本串行代码中遍历整个稀疏数组的算法思路进行并行以及优化。

首先在算法上，选择抛弃原本的按每个spot遍历的串行逻辑，选择直接对稀疏矩阵的所有非零元素进行线性化并行遍历。在并行划分上，采用针对 `for` 循环的并行策略，并在并行区域内为每个线程分配独立的足够容纳全部基因的统计数组，在每个线程计算完成后再统一归约至全局，从而得到每个基因的方差数据。

在后续的基因方差排序过程中，选择只对前 top K 个目标基因进行排序，从而进一步减少运行时间。本实验中选择筛选方差大小前 5000 的高可变基因作为目标基因。

并行实现关键代码：

```
    // 全局统计量
    std::vector<double> global_sum(n_genes_total, 0.0);
    std::vector<double> global_sum_sq(n_genes_total, 0.0);

    //并行区域
#pragma omp parallel
    {
        // 每个线程创建私有的统计数组
        std::vector<double> local_sum(n_genes_total, 0.0);
        std::vector<double> local_sum_sq(n_genes_total, 0.0);
        // 并行遍历所有非零元素 (忽略spot行结构)
#pragma omp for schedule(dynamic, 4096)
        for (int i = 0; i < total_nnz; ++i) {
            int gene_idx = sparse.indices[i];
            double val = static_cast<double>(sparse.values[i]);

            local_sum[gene_idx] += val;
            local_sum_sq[gene_idx] += val * val;
        }
        // 归约：将局部结果累加到全局结果 (加锁保护)
#pragma omp critical
        {
            for (int j = 0; j < n_genes_total; ++j) {
                global_sum[j] += local_sum[j];
                global_sum_sq[j] += local_sum_sq[j];
            }
        }
    }
    // 计算方差
    std::vector<double> variances(n_genes_total, 0.0);
    for (int gene_idx = 0; gene_idx < n_genes_total; gene_idx++) {
        double total_sum = global_sum[gene_idx];
        double total_sq = global_sum_sq[gene_idx];
        double mean = total_sum / n_spots;
        double mean_sq = total_sq / n_spots;

        // 方差
        double variance = mean_sq - (mean * mean);

        // check
        if (variance < 0 && fabs(variance) < 1e-10) {
            variance = 0.0;
        }
        else if (variance < 0) {
            printf("WARNING: Negative variance for gene %d! (value=%.5f)\n",
```

```
                gene_idx, variance);
            variance = 0.0;
        }

        variances[gene_idx] = variance;
    }

    // 排序优化：使用 partial_sort 只排前 n 个
    std::vector<int> indices(n_genes_total);
    std::iota(indices.begin(), indices.end(), 0);

    // 只要前 n_genes 个是有序的即可，后面不用管
    if (n_genes < n_genes_total) {
        std::partial_sort(indices.begin(), indices.begin() + n_genes,
indices.end(),
            [&variances](int a, int b) {
                return variances[a] > variances[b];
            });
        indices.resize(n_genes);
    }
    else {
        std::sort(indices.begin(), indices.end(), [&](int a, int b) {return
variances[a] > variances[b]; });
    }
    return indices;
```

2.2.3 并行优化过程与模块加速效果展示：

以上并行代码中的核心过程是`#pragma omp for schedule(dynamic, 4096)`，这里选择了dynamic 动态调度策略，允许线程在完成当前任务后动态申请新的任务块，可以解决因不同线程执行速度差异导致的**负载不均**问题。任务粒度，即`chunk size`则设置为4096。

以下测试结果展示了不同线程数以及不同调度策略、参数设置针对该模块的整个模块的十次测量平均结果。

不同调度方式和启动线程数测量结果展示：

	4 threads	8 threads	12 threads	16 threads	24 threads
static 默认参数	0.095327s	**0.064152s**	0.073226s	0.082700s	0.101140s
static chunk(4096)	0.095508s	**0.058181s**	0.069905s	0.078982s	0.101478s
dynamic chunk(4096)	0.109716s	0.058734s	0.052040s	0.048087s	**0.046286s**
dynamic chunk(1024)	~~未测量~~	~~未测量~~	~~未测量~~	~~未测量~~	**0.050571s**
dynamic chunk(16384)	~~未测量~~	~~未测量~~	~~未测量~~	~~未测量~~	**0.047766s**

最终选择了24 thread 进行并行的`chunk size`参数设置为 4096 的 dynamic 调度方式。很显然，实验中选择的并行线程数、调度策略以及参数设置都是会影响OpenMP的并行结果的，这和我们的认识也是一致的。

串行计时结果展示：

```
Selecting top 5000 high variance genes...

>> [Benchmark] Starting: [2]Select HVGs (Running 10 loops)...
   [Run 1] 0.519848s
   [Run 2] 0.521199s
   [Run 3] 0.512685s
   [Run 4] 0.529504s
   [Run 5] 0.506822s
   [Run 6] 0.503582s
   [Run 7] 0.497769s
   [Run 8] 0.514017s
   [Run 9] 0.537399s
   [Run 10] 0.544601s
   [Run 11] 0.517455s
-- [Result] [2]Select HVGs
   Total (without first): 5.185033s | Avg: 0.518503s
------------------------------------------------

Selected 5000 genes. Top gene ID: 14194
Feature Selection Done
```

可以看到对比串行的实验结果，在使用并行策略并进行优化后，模块的运行时间显著缩短。

2.3 矩阵转换模块

2.3.1 串行实现：

该模块的目标是将稀疏矩阵中仅包含筛选出的高变异基因（HVGs）的部分提取出来，重组为符合 PCA 输入要求的连续内存稠密矩阵。串行实现主要分为三步：

1. 构建映射关系，将筛选出的基因的原ID和稠密矩阵中的位置ID相关联。
2. CSR 遍历：按行顺序遍历稀疏矩阵的每一个 spot。
3. 查找与填充：对于每个非零元素，通过映射哈希表判断其对应的基因是否被选中；若选中，则计算一维线性偏移量 `row * width + col` 并将数据写入稠密矩阵。

串行实现关键代码：

```
DenseMatrix DenseMatrix::fromSparse(
    const SparseMatrix& sparse,
    const std::vector<int>& selected_genes) {
    const int n_spots = sparse.n_rows;
    const int n_selected = selected_genes.size();
    // 创建稠密矩阵，初始值都为0
    DenseMatrix dense(n_spots, n_selected);
    // 构建基因索引映射
    std::unordered_map<int, int> gene_to_new_index;
    for (int i = 0; i < n_selected; i++) {
        gene_to_new_index[selected_genes[i]] = i;
    }
    // 填充非零值
    for (int spot_idx = 0; spot_idx < n_spots; spot_idx++) {
        int start = sparse.indptr[spot_idx];
        int end = sparse.indptr[spot_idx + 1];
        for (int j = start; j < end; j++) {
            int gene_idx = sparse.indices[j];
            float value = sparse.values[j];
            // 检查基因是否被选中
```

```
            auto it = gene_to_new_index.find(gene_idx);
            if (it != gene_to_new_index.end()) {
                int new_col = it->second;
                dense.data_[spot_idx * n_selected + new_col] = value;
            }
            // 未选中的基因保持为0
        }
    }
    return dense;
}
```

2.3.2 并行实现：

针对以上串行代码，可以仍然使用OpenMP进行并行优化，并且实验中将原本的哈希查找表替换成了一个数组作为直接寻址表，减少了开销。仍然使用 `#pragma omp parallel for` 对外层循环进行并行化，由于稀疏矩阵中不同 spot 的非零元素数量差异较大，直接采用了 `schedule(dynamic)` 的调度策略。为了进一步加快循环的运行速度，选择将每一个 spot 的寻址步骤 `float* current_row_ptr = &dense.data_[static_cast<long long>(spot_idx) * n_selected];` 搬到循环外，从而减少了寻址时的乘法运算次数。

并行实现关键代码：

```
#include "dense_matrix.h"
#include <vector>
#include <algorithm>
#include <omp.h>

DenseMatrix DenseMatrix::fromSparse(
    const SparseMatrix& sparse,
    const std::vector<int>& selected_genes) {

    const int n_spots = sparse.n_rows;
    const int n_selected = selected_genes.size();
    const int n_total_genes = sparse.n_cols; // 总共基因num
    // 创建稠密矩阵
    DenseMatrix dense(n_spots, n_selected);
    // [优化点] 构建直接寻址表 (Lookup Table)
    // 创建一个大小为 gene num 的数组，全部填 -1
    std::vector<int> lookup_table(n_total_genes, -1);
    // 填充映射关系
    for (int i = 0; i < n_selected; i++) {
        int original_id = selected_genes[i];
        // check
        if (original_id >= 0 && original_id < n_total_genes) {
            lookup_table[original_id] = i;
        }
    }
#pragma omp parallel for schedule(dynamic, 128)
    for (int spot_idx = 0; spot_idx < n_spots; spot_idx++) {
        int start = sparse.indptr[spot_idx];
        int end = sparse.indptr[spot_idx + 1];
        // 提前算出这一spot的基地址
        float* current_row_ptr = &dense.data_[static_cast<long long>(spot_idx) *
n_selected];
        for (int j = start; j < end; j++) {
            int gene_idx = sparse.indices[j];
            float value = sparse.values[j];
```

```
            // 用数组下标直接查找
            // 只有当 gene_idx 在范围内，且对应的值不为 -1
            if (gene_idx < n_total_genes) {
                int new_col = lookup_table[gene_idx];

                if (new_col != -1) {
                    // 填入数据
                    current_row_ptr[new_col] = value;
                }
            }
        }
    }
    return dense;
}
```

2.3.3 并行优化过程与模块加速效果展示：

串行计时结果展示：

```
Converting to Dense Matrix (this may take memory)...

>> [Benchmark] Starting: [3]Sparse to Dense Conversion (Running 10 loops)...
   [Run 1] 4.034057s
   [Run 2] 5.037442s
   [Run 3] 3.587530s
   [Run 4] 3.830774s
   [Run 5] 3.495248s
   [Run 6] 3.888939s
   [Run 7] 3.544742s
   [Run 8] 3.431079s
   [Run 9] 3.524771s
   [Run 10] 3.655455s
   [Run 11] 4.164508s
-- [Result] [3]Sparse to Dense Conversion
   Total (without first): 38.160487s | Avg: 3.816049s
------------------------------------------------

Dense Matrix Created: 164978 rows x 5000 cols
Conversion Done
```

寻址优化计时结果展示：

```
Converting to Dense Matrix (this may take memory)...

>> [Benchmark] Starting: Sparse to Dense Conversion (Running 10 loops)...
   [Run 1] 1.812986s
   [Run 2] 2.906095s
   [Run 3] 2.420820s
   [Run 4] 2.197567s
   [Run 5] 2.123428s
   [Run 6] 2.195578s
   [Run 7] 2.146407s
   [Run 8] 2.016400s
   [Run 9] 1.723531s
   [Run 10] 2.047925s
   [Run 11] 1.824274s
-- [Result] Sparse to Dense Conversion
   Total (without first): 21.602024s | Avg: 2.160202s
------------------------------------------------

Dense Matrix Created: 164978 rows x 5000 cols
Conversion Done
```

可以看到在寻址优化后模块运行速度是变快的。

并行结果优化展示

测量十次时间后取均值结果：

	4 threads	8 threads	12 threads	16 threads	24 threads
(dynamic, 64)	1.578024s	1.551682s	1.568402s	1.598383s	1.553212s
(dynamic, 128)	1.765918s	1.482090s	1.502320s	1.510412s	**1.514817s**
(dynamic, 256)	1.786068s	1.740082s	1.505176s	1.526998s	1.536241s
(dynamic, 1024)	1.872292s	1.513638s	1.523117s	1.523117s	1.519483s

可以看到调整参数其实变化不大，考虑到前一个步骤中将线程数设定为 24 ，因此选择保持线程数不变，采用 `#pragma omp parallel for schedule(dynamic, 128)`。

2.4 PCA计算模块

2.4.1 串行实现：

以下展示PCA降维部分的头文件 `pca.h` ，其展示了手动使用C++实现PCA降维的计算逻辑。

```
class SerialPCA {
private:
    int n_components_;           // 主成分数 (默认50)
    std::vector<float> mean_;    // 每个基因的均值 [5000]
    std::vector<float> scale_;   // 每个基因的标准差 [5000]
    std::vector<float> components_; // 特征向量 [50 * 5000]
public:
    SerialPCA(int n_components = 50) : n_components_(n_components) {}
    // 核心函数
    void fit(const float* data, int n_samples, int n_features);
    // 降维
    void transform(const float* data, float* result,
        int n_samples, int n_features) const;
    // 获取主成分
    const std::vector<float>& getComponents() const { return components_; }

private:
    // 步骤1: 计算均值和标准差
    void compute_mean_variance(const float* data, int n_samples, int
n_features);
    // 步骤2: 标准化数据
    void standardize(float* data, int n_samples, int n_features) const;
    // 步骤3: 计算协方差矩阵
    void compute_covariance(const float* data, float* cov,
        int n_samples, int n_features) const;
    // 步骤4: 雅可比特征分解（求前k个特征向量）
    void jacobi_eigensolver(float* cov, int n_features);
};
```

可以看到 PCA主要实现放在 `.fit` 函数中，整个流程可以拆解为数据标准化、协方差矩阵计算和雅可比分解三个步骤：

1. **数据标准化**：需要遍历全量数据，计算每个基因的均值与方差，将数据归一化为均值为 0、标准差为 1 的分布。
2. **协方差矩阵计算**：基于标准化后的数据矩阵 X，计算协方差矩阵 $C = \frac{1}{N-1} X^T X$。由于 X 的维度为 $N_{spots} \times N_{genes}$，此步本质是一个大规模矩阵乘法。
3. **雅可比法实现特征值分解**：采用经典的雅可比迭代法求解实对称矩阵的特征值。通过反复寻找矩阵中绝对值最大的非对角元素，并构建旋转矩阵对其进行消除，直至所有非对角元素收敛。

结合数据规模可知，整个模块属于计算密集型，后续使用CUDA并行实现加速优化。

串行实现关键代码：

```
// 步骤1: 计算均值和标准差
void SerialPCA::compute_mean_variance(const float* data,
    int n_samples, int n_features) {
    mean_.resize(n_features, 0.0f);
    scale_.resize(n_features, 0.0f);
    // 1.1 计算均值
    for (int j = 0; j < n_features; j++) {
        float sum = 0.0f;
        for (int i = 0; i < n_samples; i++) {
            sum += data[i * n_features + j];
        }
        mean_[j] = sum / n_samples;
    }
    // 1.2 计算标准差
    for (int j = 0; j < n_features; j++) {
        float sum_sq = 0.0f;
        for (int i = 0; i < n_samples; i++) {
            float diff = data[i * n_features + j] - mean_[j];
            sum_sq += diff * diff;
        }
        scale_[j] = std::sqrt(sum_sq / (n_samples - 1));

        // check
        if (scale_[j] < 1e-8) scale_[j] = 1.0f;
    }
}

// 步骤2: 标准化数据
void SerialPCA::standardize(float* data, int n_samples, int n_features) const {
    for (int i = 0; i < n_samples; i++) {
        for (int j = 0; j < n_features; j++) {
            data[i * n_features + j] =
                (data[i * n_features + j] - mean_[j]) / scale_[j];
        }
    }
}

// 步骤3: 计算协方差矩阵
void SerialPCA::compute_covariance(const float* data, float* cov,
    int n_samples, int n_features) const {
    // 初始化协方差矩阵
    for (int i = 0; i < n_features * n_features; i++) {
        cov[i] = 0.0f;
```

```
    }
    // 计算协方差矩阵
    for (int i = 0; i < n_samples; i++) {
        const float* row = &data[i * n_features];
        for (int j = 0; j < n_features; j++) {
            for (int k = j; k < n_features; k++) { // 利用对称性
                cov[j * n_features + k] += row[j] * row[k];
            }
        }
    }
    // 除n-1、处理对称性
    float inv_n = 1.0f / (n_samples - 1);
    for (int j = 0; j < n_features; j++) {
        for (int k = j; k < n_features; k++) {
            float val = cov[j * n_features + k] * inv_n;
            cov[j * n_features + k] = val;
            cov[k * n_features + j] = val; // 对称位置
        }
    }
}

// 步骤4: 雅可比法求特征向量（只求前k个）
void SerialPCA::jacobi_eigensolver(float* cov, int n_features) {
    const int MAX_ITER = 50;
    const float EPS = 1e-6f;
    // 4.1 初始化特征向量矩阵（单位矩阵）
    components_.resize(n_components_ * n_features, 0.0f);
    std::vector<float> V(n_features * n_features, 0.0f);
    for (int i = 0; i < n_features; i++) {
        V[i * n_features + i] = 1.0f;
    }
    // 4.2 雅可比旋转迭代
    for (int iter = 0; iter < MAX_ITER; iter++) {
        // 寻找最大非对角元素
        float max_val = 0.0f;
        int p = 0, q = 1;
        for (int i = 0; i < n_features; i++) {
            for (int j = i + 1; j < n_features; j++) {
                float val = std::abs(cov[i * n_features + j]);
                if (val > max_val) {
                    max_val = val;
                    p = i; q = j;
                }
            }
        }
        if (max_val < EPS) break; // 收敛

        // 计算旋转角度
        float app = cov[p * n_features + p];
        float aqq = cov[q * n_features + q];
        float apq = cov[p * n_features + q];
        float theta = 0.5f * std::atan2(2.0f * apq, app - aqq);
        float c = std::cos(theta);
        float s = std::sin(theta);
        // 更新协方差矩阵
        float new_app = c * c * app - 2 * s * c * apq + s * s * aqq;
        float new_aqq = s * s * app + 2 * s * c * apq + c * c * aqq;
        cov[p * n_features + p] = new_app;
```

```
            cov[q * n_features + q] = new_aqq;
            cov[p * n_features + q] = cov[q * n_features + p] = 0.0f;
            for (int k = 0; k < n_features; k++) {
                if (k != p && k != q) {
                    float apk = cov[p * n_features + k];
                    float aqk = cov[q * n_features + k];
                    cov[p * n_features + k] = cov[k * n_features + p] = c * apk - s
* aqk;
                    cov[q * n_features + k] = cov[k * n_features + q] = s * apk + c
* aqk;
                }
            }
            // 更新特征向量
            for (int k = 0; k < n_features; k++) {
                float vpk = V[k * n_features + p];
                float vqk = V[k * n_features + q];
                V[k * n_features + p] = c * vpk - s * vqk;
                V[k * n_features + q] = s * vpk + c * vqk;
            }
        }

        // 4.3 提取前k个特征向量（按特征值降序）
        std::vector<std::pair<float, int>> eigen_pairs;
        for (int i = 0; i < n_features; i++) {
            eigen_pairs.push_back({ cov[i * n_features + i], i });
        }
        std::sort(eigen_pairs.begin(), eigen_pairs.end(),
            [](const auto& a, const auto& b) {
                return a.first > b.first; // 降序
            });
        // 复制前n_components_个特征向量
        for (int comp = 0; comp < n_components_; comp++) {
            int orig_idx = eigen_pairs[comp].second;
            for (int j = 0; j < n_features; j++) {
                components_[comp * n_features + j] =
                    V[j * n_features + orig_idx];
            }
        }
    }
```

2.4.2 并行实现：

（1） 数据标准化：首先针对数据标准化部分进行并行，以下两个核函数 kernel 1 和 kernel 2 分别对应了原来串行函数中的 `compute_mean_variance` 和 `standardize` 两个函数。

数据标准化并行实现：

1. kernel 1 ： Grid 中的每一个 Block 负责计算一列（一个基因特征）的统计信息，每个block中的线程采用跨步循环和共享内存规约来完成对每一列的统计。在规约步骤中采用了树状规约的结构，保证了实现速度。通过这种规划方式，相较于每一个线程处理一个列，可以启动更多的 block 数和 thread 数，从而更充分利用GPU的计算能力，实现更快的运行速度。
2. kernel 2：在kernel 1 步骤中获取了每一列的均值和方差后，kernel 2 每一个线程对应矩阵中的一个元素进行标准化。通过 `idx = blockIdx.x * blockDim.x + threadIdx.x` 计算全局线程id，对应处理矩阵中的一个元素。

核函数代码：

```
//kernel 1 统计量计算
__global__ void computeStatsKernel(const float* data, float* mean, float* scale,
int n_samples, int n_features) {
    int feature_idx = blockIdx.x; // 当前处理哪一列
    int tid = threadIdx.x;

    if (feature_idx >= n_features) return;

    double sum = 0.0;
    double sum_sq = 0.0;
    // 1. 线程并行累加：跨步循环 (Grid-Stride Loop)
    for (int i = tid; i < n_samples; i += blockDim.x) {
        float val = data[i * n_features + feature_idx];
        sum += val;
        sum_sq += val * val;
    }
    // 2. 共享内存归约
    __shared__ double s_sum[BLOCK_SIZE];
    __shared__ double s_sq[BLOCK_SIZE];
    s_sum[tid] = sum;
    s_sq[tid] = sum_sq;
    __syncthreads();
    // 树状归约
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            s_sum[tid] += s_sum[tid + stride];
            s_sq[tid] += s_sq[tid + stride];
        }
        __syncthreads();
    }
    // 3. 线程0写回并计算最终统计量
    if (tid == 0) {
        float m = (float)(s_sum[0] / n_samples);
        float s = (float)(s_sq[0] - (s_sum[0] * s_sum[0]) / n_samples);
        double var = (s_sq[0] - (s_sum[0] * s_sum[0]) / n_samples) / (n_samples
- 1);

        mean[feature_idx] = m;
        scale[feature_idx] = (var > 1e-8) ? sqrt(var) : 1.0f;
    }
}

// KERNEL 2 标准化
//每个线程处理一个元素
__global__ void standardizeKernel(float* data, const float* mean, const float*
scale, int n_elements, int n_features) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n_elements) {
        int col = idx % n_features;
        data[idx] = (data[idx] - mean[col]) / scale[col];
    }
}
```

3. 在`.fit`函数中完成数据向 GPU 的传递，以及核函数的启动。

`.fit`函数中完成数据传递：

```
// 分配显存
float* d_data, * d_mean, * d_scale, * d_cov, * d_V;
cudaMalloc(&d_data, data_size);
cudaMalloc(&d_mean, n_features * sizeof(float));
cudaMalloc(&d_scale, n_features * sizeof(float));
cudaMalloc(&d_cov, n_features * n_features * sizeof(float)); // 协方差矩阵
cudaMalloc(&d_V, n_features * n_features * sizeof(float)); // 特征向量矩阵
// 拷贝原始数据
cudaMemcpy(d_data, host_data, data_size, cudaMemcpyHostToDevice);
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
//kernel 启动
computeStatsKernel << <5000, 256 >> > (d_data, d_mean, d_scale, n_samples,
n_features);
int numBlocks = (n_samples * n_features + 256 - 1) / 256;
standardizeKernel << <numBlocks, 256 >> > (d_data, d_mean, d_scale, n_samples *
n_features, n_features);
```

kernel 1 不同并行策略的运行时间比较结果：

```
// 朴素版 Kernel 1：一个线程负责一列
__global__ void computeStatsNaiveKernel(const float* data, float* mean, float*
scale, int n_samples, int n_features) {
    // 计算当前线程负责的列
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    // check
    if (col >= n_features) return;
    double sum = 0.0;
    double sum_sq = 0.0;
    // 每个thread处理一列
    for (int row = 0; row < n_samples; row++) {
        int idx = row * n_features + col;
        float val = data[idx];
        sum += val;
        sum_sq += val * val;
    }
    // 结果并写回
    float m = (float)(sum / n_samples);
    double var = (sum_sq - (sum * sum) / n_samples) / (n_samples - 1);
    mean[col] = m;
    scale[col] = (var > 1e-8) ? sqrt(var) : 1.0f;
}
```

测量结果图如下：

```
[Step 1] Compute Mean/Var (Benchmark 10 runs):
  Run  1: 222733.703 us
  Run  2: 34401.055 us
  Run  3: 34337.250 us
  Run  4: 34382.883 us
  Run  5: 34360.227 us
  Run  6: 34359.871 us
  Run  7: 34398.078 us
  Run  8: 34356.930 us
  Run  9: 34409.438 us
  Run 10: 34332.125 us
  Run 11: 34380.930 us
[kernel 1] Average Time: 34371.879 us
```

优化后的 kernel 1 （即上面展示的并行实现版）测量结果如下：

```
[Step 1] Compute Mean/Var (Benchmark 10 runs):
  Run  1: 32649.441 us
  Run  2: 30215.617 us
  Run  3: 30214.559 us
  Run  4: 30089.279 us
  Run  5: 29917.984 us
  Run  6: 29993.471 us
  Run  7: 30173.377 us
  Run  8: 30073.281 us
  Run  9: 30052.480 us
  Run 10: 29893.248 us
  Run 11: 29946.592 us
[kernel 1] Average Time: 30056.984 us
```

可以看到启动更多的block以及thread的版本具有更好的运行速度。

（2）协方差矩阵计算：该部分对应了串行代码中的 `compute_covariance` 函数，采用的并行策略是每个线程处理输出矩阵的一个位置的输出，并且采用二维网格映射，将输出矩阵划分成不同的方块分配给 block 处理。由于协方差矩阵是对称矩阵，因此为了加快计算速度，强制线程仅计算上三角矩阵减少运算次数。

核函数代码：

```
// KERNEL 3: 协方差矩阵计算
// 策略：每个线程计算 Cov[row][col] 的一个元素。
__global__ void computeCovarianceKernel(const float* data, float* cov, int
n_samples, int n_features) {
    // 输出矩阵的坐标 (row, col)
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < n_features && col < n_features) {
        // 利用对称性，只计算上三角 (row <= col)
        if (row <= col) {
            double sum = 0.0;
            // 计算两列的点积
            for (int k = 0; k < n_samples; k++) {
                float val1 = data[k * n_features + row];
                float val2 = data[k * n_features + col];
                sum += val1 * val2;
            }
            float res = (float)(sum / (n_samples - 1));
```

```
            cov[row * n_features + col] = res;
            cov[col * n_features + row] = res; // 对称赋值
        }
    }
}
```

`.fit`函数中完成核函数的调用：

```
    dim3 dimBlock(16, 16);
    dim3 dimGrid((n_features + 15) / 16, (n_features + 15) / 16);
    for (int i = 0; i < iterations ; i++) {
        cudaEventRecord(start);

        // 启动 Kernel
        computeCovarianceKernel << <dimGrid, dimBlock >> > (d_data, d_cov,
n_samples, n_features);
```

事实上关于每个 block 中的 thread 分配，尝试了不同的设置试图进一步加速，考虑到每个 warp 有 32 个线程，因此尝试将 thread 设置为 （32，8）、（32，16）、（32，32）、（16，32）、（8，32）进行时间测量，所得结果如下表。

	(16, 16)	**(32, 8)**	**(32, 16)**	**(32, 32)**	**(16, 32)**	**(8, 32)**
测量结果	**32.225 s**	32.665 s	32.802 s	33.017 s	32.500 s	32.369 s

实验显示，调整参数运行时间变化不大，因此最后仍然选定每个 block 含有 16 * 16 thread。

（3）雅可比法实现特征值分解：该分解过程可以继续拆分为搜索最大非对角元素、旋转矩阵、筛选特征值和对应特征向量三个部分。

最大非对角元素搜索

```
// 最大非对角元素搜索host端代码

size_t shared_mem_size = BLOCK_SIZE * sizeof(float) + 2 * BLOCK_SIZE *
sizeof(int);
findMaxOffDiagKernel << <reduce_blocks, BLOCK_SIZE, shared_mem_size >> > (d_cov,
d_max_vals, d_p, d_q, n_features);
// 拷回 CPU 做最后一步归约 (因为 block 数很少，CPU 做这一步更快)
cudaMemcpy(h_max_vals.data(), d_max_vals, reduce_blocks * sizeof(float),
cudaMemcpyDeviceToHost);
cudaMemcpy(h_p.data(), d_p, reduce_blocks * sizeof(int),
cudaMemcpyDeviceToHost);
cudaMemcpy(h_q.data(), d_q, reduce_blocks * sizeof(int),
cudaMemcpyDeviceToHost);
float max_val = 0.0f;
int p = 0, q = 1;
for (int i = 0; i < reduce_blocks; ++i) {
    if (h_max_vals[i] > max_val) {
        max_val = h_max_vals[i];
        p = h_p[i];
        q = h_q[i];
    }
}
```

```
if (max_val < 1e-6f) break; // 收敛
```

```
// 最大非对角元素搜索kernel

__global__ void findMaxOffDiagKernel(const float* cov, float* d_max_vals, int*
d_p, int* d_q, int n_features) {
    // 动态共享内存
    extern __shared__ float s_max_val[];
    int* s_p = (int*)&s_max_val[blockDim.x]; // 共享内存布局技巧
    int* s_q = (int*)&s_p[blockDim.x];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // 总元素个数
    float local_max = -1.0f;
    int local_p = 0;
    int local_q = 1;
    // 跨步搜索
    int total_elements = n_features * n_features;
    for (int i = idx; i < total_elements; i += gridDim.x * blockDim.x) {
        int r = i / n_features;
        int c = i % n_features;
        if (c > r) { // 只看上三角且非对角
            float val = fabsf(cov[i]);
            if (val > local_max) {
                local_max = val;
                local_p = r;
                local_q = c;
            }
        }
    }
    // 存入 Shared Memory
    s_max_val[tid] = local_max;
    s_p[tid] = local_p;
    s_q[tid] = local_q;
    __syncthreads();
    // Block 内归约找最大
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            if (s_max_val[tid + stride] > s_max_val[tid]) {
                s_max_val[tid] = s_max_val[tid + stride];
                s_p[tid] = s_p[tid + stride];
                s_q[tid] = s_q[tid + stride];
            }
        }
        __syncthreads(); //规约一定要保证同步
    }
    // 写回 Block 结果
    if (tid == 0) {
        d_max_vals[blockIdx.x] = s_max_val[0];
        d_p[blockIdx.x] = s_p[0];
        d_q[blockIdx.x] = s_q[0];
    }
}
```

针对非对角元素搜索的阶段，采用了 GPU 和 CPU 混合规约的方式并行化找到目标元素。GPU 中每个 block 设置为 256 个线程，共设置 256 个 block。每个线程除了处理自身全局 ID 对应位置的数据之外还会向后跨步搜索，直到超出协方差矩阵边界，比较得到每个线程负责数据中的最大值。成功遍历协方差矩阵之后，每个 block 内部使用规约，筛选出每个块对应的一个最大值。由于设置了 256 个 block，这时共有 256 个数据需要排序。选择将数据传回 CPU 快速实现排序，从而得到整个协方差矩阵的非对角最大值和对应位置。针对每个 block 使用了共享内存，大小为：`线程块大小 × (1个浮点数 + 2个整数)`，分别用于存储最大值、行坐标 p 和列坐标 q。除了用共享内存加速块内对比，还是用了之前使用过的树状规约，进一步加快运行速度。

旋转矩阵

```
//旋转矩阵部分host端代码

// 手动更新对角线元素 (为了减少开销，这几个数直接算好填进去)
float new_app = c * c * app - 2 * s * c * apq + s * s * aqq;
float new_aqq = s * s * app + 2 * s * c * apq + c * c * aqq;
float zero = 0.0f;
cudaMemcpy(&d_cov[p * n_features + p], &new_app, sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(&d_cov[q * n_features + q], &new_aqq, sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(&d_cov[p * n_features + q], &zero, sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(&d_cov[q * n_features + p], &zero, sizeof(float),
cudaMemcpyHostToDevice);
// 执行旋转更新其他元素 (Kernel 5)
int rot_blocks = (n_features + BLOCK_SIZE - 1) / BLOCK_SIZE;
jacobiRotateKernel << <rot_blocks, BLOCK_SIZE >> > (d_cov, d_V, p, q, c, s,
n_features);
}
```

```
// KERNEL 5 旋转更新kernel
// 并行更新 Cov 矩阵和特征向量矩阵 V
__global__ void jacobiRotateKernel(float* cov, float* V, int p, int q, float c,
float s, int n_features) {
    int k = blockIdx.x * blockDim.x + threadIdx.x;
    if (k >= n_features) return;
    // 更新 Cov (只更新第 p 行/列 和 第 q 行/列)
    if (k != p && k != q) {
        float cov_pk = cov[p * n_features + k];
        float cov_qk = cov[q * n_features + k];
        // 公式参考串行代码
        float new_cov_pk = c * cov_pk - s * cov_qk;
        float new_cov_qk = s * cov_pk + c * cov_qk;
        cov[p * n_features + k] = new_cov_pk;
        cov[k * n_features + p] = new_cov_pk; // 对称
        cov[q * n_features + k] = new_cov_qk;
        cov[k * n_features + q] = new_cov_qk; // 对称
    }

    // 更新特征向量 V (所有行都需要更新)
    float v_kp = V[k * n_features + p]; // V[k][p]
    float v_kq = V[k * n_features + q]; // V[k][q]
    V[k * n_features + p] = c * v_kp - s * v_kq;
    V[k * n_features + q] = s * v_kp + c * v_kq;
}
```

在前一步搜索得到最大非对角元素之后，直接在 CPU 上完成计算旋转角度 θ 以及对应的旋转参数 $\cos\theta$ 和 $\sin\theta$，并直接计算更新对角线上的主元素 Cov_{pp}, Cov_{qq}。这样可以避免启用 kernel 造成多余开销。

获得旋转参数后，启动对核函数对协方差矩阵的其他相关元素和特征向量矩阵进行更新。由于旋转仅影响第 p 行/列和第 q 行/列，并行策略为每个线程负责计算一个列索引 k 下的新值（即 Cov_{pk} 和 Cov_{qk}）。针对特征向量矩阵的更新，并行策略和旋转一致。

筛选特征值和对应向量

```
//host端获得所需要位数的特征值和对应的特征向量
std::vector<float> h_cov_full(n_features * n_features);
cudaMemcpy(h_cov_full.data(), d_cov, n_features * n_features * sizeof(float),
cudaMemcpyDeviceToHost);
for (int i = 0; i < n_features; ++i) h_diag[i] = h_cov_full[i * n_features + i];

cudaMemcpy(h_V_full.data(), d_V, n_features * n_features * sizeof(float),
cudaMemcpyDeviceToHost);
```

前一步旋转过程收敛后，将变换后的协方差矩阵和特征向量矩阵从 device 端转移回 host 端，在 CPU 上排序选择出所需要的特征值对应的特征向量，整个 PCA 计算模块的并行化完整实现了。

2.4.3 并行模块加速效果展示：

串行版本 PCA 计算计时结果：

```
Running PCA fit (calculating covariance & eigenvectors)...
WARNING: Since we use Jacobi method, this step might be slow for 5000 genes.
>> [Benchmark] Starting: [4]PCA Fit (Covariance + Jacobi) (Running 1 loops)...
   [Run 1] 1859.784256s
-- [Result] [4]PCA Fit (Covariance + Jacobi)
   Total (without first): 1859.784256s | Avg: 1859.784256s
```

并行版本 PCA 计算计时结果：

```
-- [Result] [4]PCA Fit (Covariance + Jacobi)
   Total (without first): 33.275002s | Avg: 33.275002s
------------------------------------------------
```

可以看到使用 CUDA 并行化处理计算密集型部分，可以获得很高的加速比。

2.5 数据投影：

2.5.1 串行实现：

在得到所需要的 PCA 维度对应的特征向量以后，只需要将原本的 spots * genes 的数据矩阵和之前拟合时一样归一化，然后投影到这 50 个特征向量组成的空间即可，写成数学形式即为 $R = X_{std} \cdot V^T$。

串行实现关键代码：

```
// 降维：投影到主成分
void SerialPCA::transform(const float* data, float* result,
    int n_samples, int n_features) const {
    // 标准化新数据
    std::vector<float> data_std(n_samples * n_features);
    for (int i = 0; i < n_samples; i++) {
        for (int j = 0; j < n_features; j++) {
            int idx = i * n_features + j;
            data_std[idx] = (data[idx] - mean_[j]) / scale_[j];
```

```
        }
    }
    // 矩阵乘法: result = data_std * components_^T
    for (int i = 0; i < n_samples; i++) {
        for (int k = 0; k < n_components_; k++) {
            float sum = 0.0f;
            for (int j = 0; j < n_features; j++) {
                sum += data_std[i * n_features + j] *
                    components_[k * n_features + j];
            }
            result[i * n_components_ + k] = sum;
        }
    }
}
```

2.5.2 并行实现：

直接使用了朴素的矩阵乘法并行，每个线程通过全局索引 `(row, col)` 负责计算对应的第 `row` 个样本在第 `col` 个主成分上的投影值，即使用 `Data` 的第 i 行与 `Components` 的第 j 行进行点积运算。

并行实现关键代码：

```
// 矩阵乘法核函数
__global__ void matrixMulKernel(const float* data, const float* components,
float* result,
    int n_samples, int n_features, int n_components) {
    int row = blockIdx.y * blockDim.y + threadIdx.y; // Sample index
    int col = blockIdx.x * blockDim.x + threadIdx.x; // Component index

    if (row < n_samples && col < n_components) {
        double sum = 0.0;
        // 点积：Data的第row行 dot Components的第col行
        for (int k = 0; k < n_features; k++) {
            float d = data[row * n_features + k];
            float c = components[col * n_features + k];
            // components 存储是 [n_comp][n_feat]
            sum += d * c;
        }
        result[row * n_components + col] = (float)sum;
    }
}
```

2.5.3 并行模块加速效果展示：

串行版本数据投影计时结果：

```
------------------------------------------------
PCA Fit DoneProjecting data to 50 PCs...
>> [Benchmark] Starting: [5]PCA Transform (Running 1 loops)...
   [Run 1] 34.992965s
-- [Result] [5]PCA Transform
   Total (without first): 34.992965s | Avg: 34.992965s
------------------------------------------------
```

并行版本数据投影计时结果：

```
------------------------------------------------
PCA Fit DoneProjecting data to 50 PCs...
>> [Benchmark] Starting: [5]PCA Transform (Running 1 loops)...
   [Run 1] 2.386064s
-- [Result] [5]PCA Transform
   Total (without first): 2.386064s | Avg: 2.386064s
------------------------------------------------
```

获得 CUDA 并行后获得了显著的加速效果。

2.6 K-Means聚类：

2.6.1 串行实现：

该模块负责对 PCA 降维后的数据进行无监督聚类，以识别具有相似表达模式的 spots 。在经过之前的 PCA 降维处理后，目前矩阵的行代表样本，每一列代表一个主成分。K-Means 的具体计算流程如下：

1. **初始化 (Initialization)**：为了保证实验的可复现性，使用固定种子的随机数生成器（`std::mt19937`），从数据集中随机选取 K 个样本点作为初始质心。在串行代码中对应函数 `initializeCentroids` 。本实验中 K 设定为 10 。
2. **指派步骤 (E-Step)**：遍历所有 N_{spots} 个样本，计算每个样本到 K 个质心的欧氏距离平方。通过比较距离平方值，将样本分配到距离最近的簇中。在串行代码中对应函数 `computeAssignments` 。
3. **更新步骤 (M-Step)**：遍历所有样本，根据其所属的簇标签，累加对应的坐标值并统计簇内样本数。遍历完成后，通过 `Sum / Count` 计算新的几何中心作为下一轮的质心。在串行代码中对应函数 `updateCentroids`。
4. **收敛判决**：计算新旧质心之间的总平方位移。若位移小于设定的阈值 `tol`，则判定为收敛提前终止迭代，避免不必要的时间浪费。判定部分放在主要实现函数 `.fit` 中进行，和之前的 PCA 实现方式是类似的。

串行实现关键代码：

```cpp
void KMeans::initializeCentroids(const std::vector<float>& pca_data) {
    int dim = n_components; // 维度就是 PCA 的维度
    centroids.assign(k * dim, 0.0f);
    assignments.resize(n_spots);
    distances.resize(n_spots);
    // 随机选择 k 个点作为初始中心
    std::vector<int> init_indices(k);
    const unsigned int FIXED_SEED = 7; // 使用固定的种子
    std::mt19937 gen(FIXED_SEED);
    std::uniform_int_distribution<> dis(0, n_spots - 1);
    for (int i = 0; i < k; i++) {
        init_indices[i] = dis(gen);
    }
    // 复制初始中心
    for (int i = 0; i < k; i++) {
        int idx = init_indices[i];
        int src_offset = idx * n_components;
        int dst_offset = i * dim;
        // 内存拷贝
        std::copy(pca_data.begin() + src_offset,
            pca_data.begin() + src_offset + n_components,
            centroids.begin() + dst_offset);
    }
}

// 计算分配 (E-step)
```

```cpp
void KMeans::computeAssignments(const std::vector<float>& pca_data) {
    int dim = n_components;
    for (int idx = 0; idx < n_spots; idx++) {
        float best_dist = std::numeric_limits<float>::max();
        int best_cluster = 0;
        // 当前点的 PCA 数据起始位置
        int data_offset = idx * n_components;
        for (int c = 0; c < k; c++) {
            float dist = 0.0f;
            int center_offset = c * dim;
            // 计算纯 PCA 特征的欧氏距离平方
            for (int j = 0; j < n_components; j++) {
                float val = pca_data[data_offset + j];
                float center_val = centroids[center_offset + j];
                float diff = val - center_val;
                dist += diff * diff;
            }
            // 更新最近簇
            if (dist < best_dist) {
                best_dist = dist;
                best_cluster = c;
            }
        }
        assignments[idx] = best_cluster;
        distances[idx] = best_dist; // 虽然这里存的是平方距离，但不影响比较
    }
}

// 更新质心并计算移动距离方便（提前退出
float KMeans::updateCentroids(const std::vector<float>& pca_data) {
    int dim = n_components;
    // 备份旧质心，并清空当前向量
    std::vector<float> old_centroids = centroids;
    std::fill(centroids.begin(), centroids.end(), 0.0f);
    std::vector<int> counts(k, 0);
    // 累加所有点的数据
    for (int idx = 0; idx < n_spots; idx++) {
        int cluster = assignments[idx];
        counts[cluster]++;
        int center_offset = cluster * dim;
        int data_offset = idx * n_components;
        for (int j = 0; j < n_components; j++) {
            centroids[center_offset + j] += pca_data[data_offset + j];
        }
    }
    // 计算平均值（得到新质心）
    for (int c = 0; c < k; c++) {
        int count = counts[c];
        if (count > 0) {
            int center_offset = c * dim;
            float inv_count = 1.0f / count;
            for (int j = 0; j < dim; j++) {
                centroids[center_offset + j] *= inv_count;
            }
        }
    }
    // 计算质心总位移（shift）
    float total_shift = 0.0f;
```

```
    for (int i = 0; i < k * dim; i++) {
        float diff = centroids[i] - old_centroids[i];
        total_shift += diff * diff; // 使用平方和，避免开根号更高效
    }
    return total_shift; // 返回总平方位移
}

std::vector<int> KMeans::fit(const std::vector<float>& pca_data, int n_comps) {
    n_components = n_comps;
    if (n_components == 0) return {};
    n_spots = pca_data.size() / n_components;
    std::cout << "Running KMeans (k=" << k << ") with convergence check..." <<
std::endl;
    initializeCentroids(pca_data);
    for (int iter = 0; iter < max_iter; iter++) {
        // E-Step
        computeAssignments(pca_data);
        // M-Step (获取移动距离)
        float shift = updateCentroids(pca_data);
        // 如果质心移动非常小 则收敛提前结束
        if (shift < tol) {
            std::cout << "  Converged at iteration " << iter + 1
                << " (Shift: " << shift << " < " << tol << ")" << std::endl;
            break;
        }
    }
    std::cout << "Clustering done." << std::endl;
    return assignments;
}
```

2.6.2 并行实现：

E step并行化

并行策略是每个 thread 处理一个样本点（spot），在计算中，每个线程遍历所有 K 个质心，计算欧氏距离平方并确定最近簇标签。该并行策略会消除了串行算法中最外层的样本遍历循环，并利用 GPU 的计算能力极大提升运算速度。除此之外，由于各个线程处理数据的时候都需要读取质心的信息，可以从内存优化的角度考虑进行进一步优化。由于常量内存大小非常小，共享内存又是针对每个 block 的，因此在这里采用只读缓存修饰符 `__restrict__`，从而启用 L1 Cache 缓存质心数据，减少了对全局内存的重复读取。

E step并行实现核心代码：

```
// E kernel
__global__ void computeAssignmentsKernel(
    const float* __restrict__ data,      // PCA 数据
    const float* __restrict__ centroids, // 质心数据
    int* __restrict__ assignments,       //label结果
    float* __restrict__ distances,       // 距离平方
    int n_spots,
    int n_components,
    int k
) {
    // 计算当前线程负责的样本索引
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n_spots) return;
    float best_dist = 3.402823466e+38F; // float max
```

```
    int best_cluster = 0;
    // 数据在全局内存中的起始偏移量
    int data_offset = idx * n_components;
    // 遍历所有质心，每个线程对每个sopt计算对全部代表点的距离平方
    for (int c = 0; c < k; c++) {
        float dist = 0.0f;
        int center_offset = c * n_components;
        for (int j = 0; j < n_components; j++) {
            float val = data[data_offset + j];
            float center_val = centroids[center_offset + j];
            float diff = val - center_val;
            dist += diff * diff;
        }
        // 更新最近簇
        if (dist < best_dist) {
            best_dist = dist;
            best_cluster = c;
        }
    }
    assignments[idx] = best_cluster;
    distances[idx] = best_dist;
}

// E-step cuda版本
void KMeans::computeAssignments(float* d_data, float* d_centroids, int*
d_assignments, float* d_distances) {
    //配置 Kernel 参数
    int threadsPerBlock = 256;
    int blocksPerGrid = (n_spots + threadsPerBlock - 1) / threadsPerBlock;
    computeAssignmentsKernel << <blocksPerGrid, threadsPerBlock >> > (
        d_data,
        d_centroids,
        d_assignments,
        d_distances,
        n_spots,
        n_components,
        k
        );
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("Kernel Launch Error: %s\n", cudaGetErrorString(err));
    }
    cudaDeviceSynchronize();
}
```

M step并行化

该步骤中利用 GPU 并行执行新质心的累加统计，每个线程处理一个数据点，每个线程独立查阅样本点的簇标签，并计算出数据坐标 `data_offset` 以及需要存入的位置坐标 `center_offset` 。面对多线程同时写入同一簇累加器的竞争问题，采用了 全局原子操作 （ `atomicAdd` ），最终实现将坐标值累加到全局显存中的 `new_centroids_sum` 数组。

在GPU上完成核函数的执行后，带有 PCA 数值的标签数组不会被传回 CPU ，只有极小规模的“累加和”与“计数器”数组传回 CPU 进行最终的除法平均和位移计算，这减小了数据传输的时间消耗，提升了运行速度。

M step并行实现核心代码:

```
// M-step Kernel
__global__ void updateCentroidsAccumulateKernel(
    const float* __restrict__ data,
    const int* __restrict__ assignments,
    float* __restrict__ new_centroids_sum,
    int* __restrict__ cluster_counts,
    int n_spots,
    int n_components,
    int k
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n_spots) return;
    // 获取当前点所属的簇
    int cluster_id = assignments[idx];
    // 原子操作防止冲突
    atomicAdd(&cluster_counts[cluster_id], 1);
    // 累加坐标
    int data_offset = idx * n_components;
    int center_offset = cluster_id * n_components;
    for (int j = 0; j < n_components; j++) {
        float val = data[data_offset + j];
        //同样用原子操作
        atomicAdd(&new_centroids_sum[center_offset + j], val);
    }
}

// 更新质心并计算移动距离方便（提前推出
float KMeans::updateCentroids(float* d_data, int* d_assignments, float*
d_centroids_sum, int* d_counts) {
    int dim = n_components;
    // 每次迭代前，把用于累加的新质心内存和计数器清零
    cudaMemset(d_centroids_sum, 0, k * dim * sizeof(float));
    cudaMemset(d_counts, 0, k * sizeof(int));
    // 启动累加 Kernel
    int threadsPerBlock = 256;
    int blocksPerGrid = (n_spots + threadsPerBlock - 1) / threadsPerBlock;
    updateCentroidsAccumulateKernel << <blocksPerGrid, threadsPerBlock >> > (
        d_data,
        d_assignments,
        d_centroids_sum,
        d_counts,
        n_spots,
        dim,
        k
        );
    cudaDeviceSynchronize();
    // 拷贝 k*dim 个数据
    std::vector<float> h_new_centroids_sum(k * dim);
    std::vector<int> h_counts(k);
    cudaMemcpy(h_new_centroids_sum.data(), d_centroids_sum, k * dim *
sizeof(float), cudaMemcpyDeviceToHost);
    cudaMemcpy(h_counts.data(), d_counts, k * sizeof(int),
cudaMemcpyDeviceToHost);
    // 计算这次的质心和上次的距离
    float total_shift = 0.0f;
    // 备份旧质心
    std::vector<float> old_centroids = centroids;
    for (int c = 0; c < k; c++) {
```

```
        int count = h_counts[c];
        int offset = c * dim;
        if (count > 0) {
            float inv_count = 1.0f / count;
            for (int j = 0; j < dim; j++) {
                // 计算平均值：累加和 / 计数
                centroids[offset + j] = h_new_centroids_sum[offset + j] *
inv_count;
            }
        }
    }
    for (int i = 0; i < k * dim; i++) {
        float diff = centroids[i] - old_centroids[i];
        total_shift += diff * diff;
    }
    return total_shift;
}
```

`.fit` 函数核心部分

```
// pca_data 只拷贝一次，一直在显存中
cudaMemcpy(d_data, pca_data.data(), data_size, cudaMemcpyHostToDevice);
for (int iter = 0; iter < max_iter; iter++) {
    // E
    cudaMemcpy(d_centroids, centroids.data(), centroids_size,
cudaMemcpyHostToDevice);//小数据
    computeAssignments(d_data, d_centroids, d_assignments, d_distances);
    float shift = updateCentroids(d_data, d_assignments, d_centroids_sum,
d_counts);
    // 收敛判定
    // 如果质心移动非常小则提前结束
    if (shift < tol) {
        std::cout << "  Converged at iteration " << iter + 1
            << " (Shift: " << shift << " < " << tol << ")" << std::endl;
        break;
    }
}
//标签回传
cudaMemcpy(assignments.data(), d_assignments, assign_size,
cudaMemcpyDeviceToHost);
```

可以看到代码将规模较大的数据 `pca_data` 的拷贝过程放在循环外，避免了在迭代循环中反复拷贝大规模数据，在循环内只传输一些小规模数据，从而减少模块的运行时间。

2.6.3 并行模块加速效果展示：

串行版本 K-Means 计时结果：

```
Clustering done.
   [Run 1] 3.157419s
-- [Result] [6]KMeans Clustering
   Total (without first): 3.157419s | Avg: 3.157419s
------------------------------------------------
```

并行版本K-Means 计时结果：

```
Clustering done.
   [Run 1] 0.105717s
-- [Result] [6]KMeans Clustering
   Total (without first): 0.105717s | Avg: 0.105717s
------------------------------------------------
```

可以看到并行处理的K-Means 显著加速。

3.实验结果与分析

3.1 展示完整流程串行和并行运行

串行完整输出与计时:

```
Loading sparse matrix from binary...

>> [Benchmark] Starting: Load Sparse Matrix (Running 1 loops)...
   [Run 1] 2.842334s
-- [Result] Load Sparse Matrix
   Total (without first): 2.842334s | Avg: 2.842334s
------------------------------------------------

Loaded: 164978 spots x 18000 genes (NNZ: 176609566)
Data Loaded
Selecting top 5000 high variance genes...

>> [Benchmark] Starting: [2]Select HVGs (Running 1 loops)...
   [Run 1] 0.782428s
-- [Result] [2]Select HVGs
   Total (without first): 0.782428s | Avg: 0.782428s
------------------------------------------------

Selected 5000 genes. Top gene ID: 14194
Feature Selection Done
Converting to Dense Matrix (this may take memory)...

>> [Benchmark] Starting: [3]Sparse to Dense Conversion (Running 1 loops)...
   [Run 1] 5.089755s
-- [Result] [3]Sparse to Dense Conversion
   Total (without first): 5.089755s | Avg: 5.089755s
------------------------------------------------

Dense Matrix Created: 164978 rows x 5000 cols
Conversion Done
Running PCA fit (calculating covariance & eigenvectors)...
WARNING: Since we use Jacobi method, this step might be slow for 5000 genes.
>> [Benchmark] Starting: [4]PCA Fit (Covariance + Jacobi) (Running 1 loops)...
   [Run 1] 1859.784256s
-- [Result] [4]PCA Fit (Covariance + Jacobi)
   Total (without first): 1859.784256s | Avg: 1859.784256s
------------------------------------------------
```

```
------------------------------------------------
PCA Fit DoneProjecting data to 50 PCs...
>> [Benchmark] Starting: [5]PCA Transform (Running 1 loops)...
   [Run 1] 34.992965s
-- [Result] [5]PCA Transform
   Total (without first): 34.992965s | Avg: 34.992965s
------------------------------------------------
PCA Transform Done
Running KMeans Clustering...
>> [Benchmark] Starting: [6]KMeans Clustering (Running 1 loops)...
Running KMeans (k=10) with convergence check...
  Converged at iteration 40 (Shift: 9.62728e-05 < 0.0001)
Clustering done.
   [Run 1] 3.157419s
-- [Result] [6]KMeans Clustering
   Total (without first): 3.157419s | Avg: 3.157419s
------------------------------------------------
KMeans Clustering Done   [ total time ] 1906.667033s
Pipeline Finished
```

并行完整输出与计时：

```
Loading sparse matrix from binary...

>> [Benchmark] Starting: Load Sparse Matrix (Running 1 loops)...
   [Run 1] 1.352334s
-- [Result] Load Sparse Matrix
   Total (without first): 1.352334s | Avg: 1.352334s
------------------------------------------------

Loaded: 164978 spots x 18000 genes (NNZ: 176609566)
Data Loaded
Selecting top 5000 high variance genes...

>> [Benchmark] Starting: Select HVGs (24 threads) (Running 1 loops)...
   [Run 1] 0.048399s
-- [Result] Select HVGs (24 threads)
   Total (without first): 0.048399s | Avg: 0.048399s
------------------------------------------------

Selected 5000 genes. Top gene ID: 14194
Feature Selection Done
Converting to Dense Matrix (this may take memory)...

>> [Benchmark] Starting: Sparse to Dense Conversion (24 threads) (Running 1 loops)...
   [Run 1] 1.447950s
-- [Result] Sparse to Dense Conversion (24 threads)
   Total (without first): 1.447950s | Avg: 1.447950s
------------------------------------------------

Dense Matrix Created: 164978 rows x 5000 cols
Conversion Done
Running PCA fit (calculating covariance & eigenvectors)...
WARNING: Since we use Jacobi method, this step might be slow for 5000 genes.
>> [Benchmark] Starting: [4]PCA Fit (Covariance + Jacobi) (Running 1 loops)...
  [kernel 1] :
    Run  1: 32980.000 us
  [kernel 2] :
    Run  1: 20885.920 us
```

```
  [kernel 3] :
    Run  1: 32141192.000 us
  [kernel 3] Average Time: 32141192.000 us

Starting Jacobi Iteration (Manual CUDA)...
   [jacobi_entire_time 1] 0.066864s
   [Run 1] 33.275002s
-- [Result] [4]PCA Fit (Covariance + Jacobi)
   Total (without first): 33.275002s | Avg: 33.275002s
------------------------------------------------
PCA Fit DoneProjecting data to 50 PCs...
>> [Benchmark] Starting: [5]PCA Transform (Running 1 loops)...
   [Run 1] 2.386064s
-- [Result] [5]PCA Transform
   Total (without first): 2.386064s | Avg: 2.386064s
------------------------------------------------
```

```
Clustering done.
   [Run 1] 0.105717s
-- [Result] [6]KMeans Clustering
   Total (without first): 0.105717s | Avg: 0.105717s
------------------------------------------------
KMeans Clustering Done   [ total time ] 38.629753s
```

	数据加载	基因筛选	矩阵转换	PCA 计算	数据投影	K-Means 聚类	完整流程
串行	2.842334s	0.782428s	5.089755s	1859.784256s	34.992965s	3.157419s	1906.667033s
并行	1.352334s	0.048399s	1.447950s	33.275002s	2.386064s	0.105717s	38.629753s

可以计算出整个流程的加速比为 **49.36**。

3.2 结果分析

可以看到原本串行过程中的核心限速步骤其实是PCA 模块，而 K-means 步骤由于在进行之前已经完成了降维，使得数据大小显著降低，因此没有成为流程的限速步骤。如果想要进一步显著提高加速比，需要尝试针对协方差的计算过程做更加细致的优化，由于本实验，从串行实现到并行实现，代码工程量大，花费较多时间，因此暂时没有进一步尝试优化。

3.3 总结

本项目针对大规模空间转录组数据分析流程，使用C++ 手动实现了一套串行代码。并且在此基础上设计并实现了一套基于 OpenMP 与 CUDA 的混合并行加速系统。通过对数据加载、高变基因筛选、稀疏矩阵转换、PCA 降维及 K-Means 聚类等核心模块的深度优化，显著降低了端到端运行时间。

4. 附录

4.1 时间记录函数

```
//一个简单的多轮次计时函数,可以调整决定是否预热
void benchmark(const std::string& name, int iterations, std::function<void()>
func) {
    printf(">> [Benchmark] Starting: %s (Running %d loops)...\n", name.c_str(),
iterations);
    double total_execution_time = 0.0;
    for (int i = 0; i < iterations; ++i) {
        auto start = std::chrono::high_resolution_clock::now();
        func();
        auto end = std::chrono::high_resolution_clock::now();

        std::chrono::duration<double> elapsed = end - start;
        double current_run_time = elapsed.count();
        if (i > -1) {
            total_execution_time += current_run_time;
        }
        printf("   [Run %d] %.6fs\n", i + 1, current_run_time);
    }
    double avg_time = total_execution_time / iterations;
    printf("-- [Result] %s\n", name.c_str());
    printf("   Total (without first): %.6fs | Avg: %.6fs\n",
total_execution_time, avg_time);
    printf("------------------------------------------------\n");
}
```

最终测量加速比时每个模块采用以上函数计时，并且只运行一次记录时间。全流程时间记录则在 `main()` 函数中加入 `std::chrono::high_resolution_clock::now()` 计时。

4.2 源代码

串行实现和并行实现的两份源代码随报告一并提交。